

Apuntes – Clase 7

Memoria virtual

Sistemas Operativos

Universidad de Antioquia

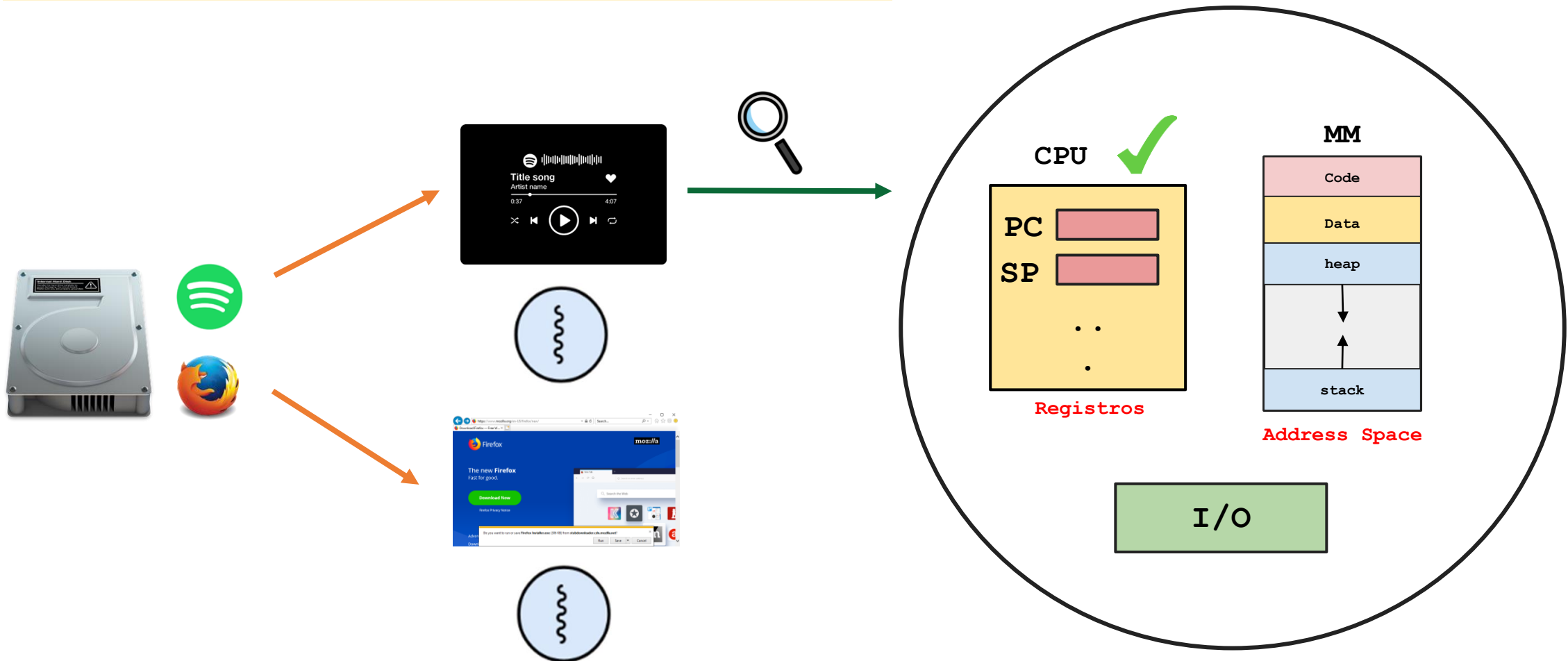
Facultad de Ingeniería

Ingeniería de Sistemas

Proceso

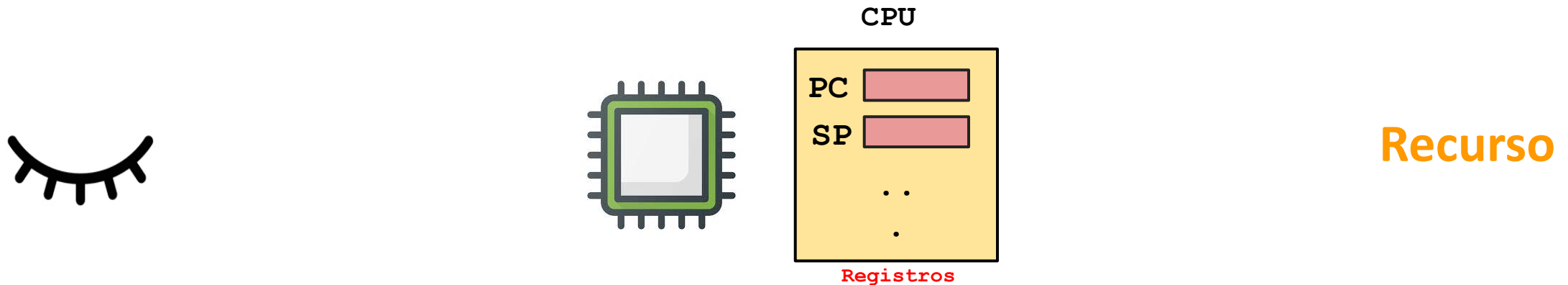
Un proceso es un **programa en ejecución**.

Proceso = CPU + Memory + I/O Info



Virtualización de CPU

Cada proceso cree que **tiene su propia CPU**.



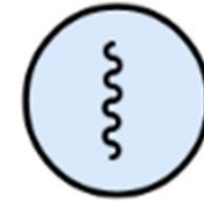
Implementación de la Virtualización

¿Cómo se implementa la virtualización?

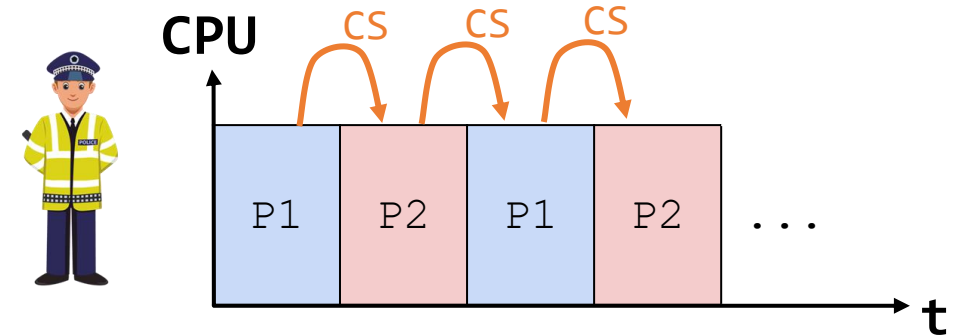
Process management



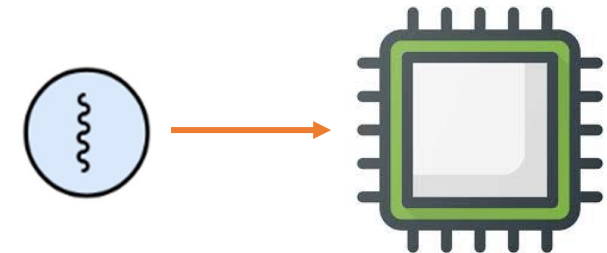
Abstracción



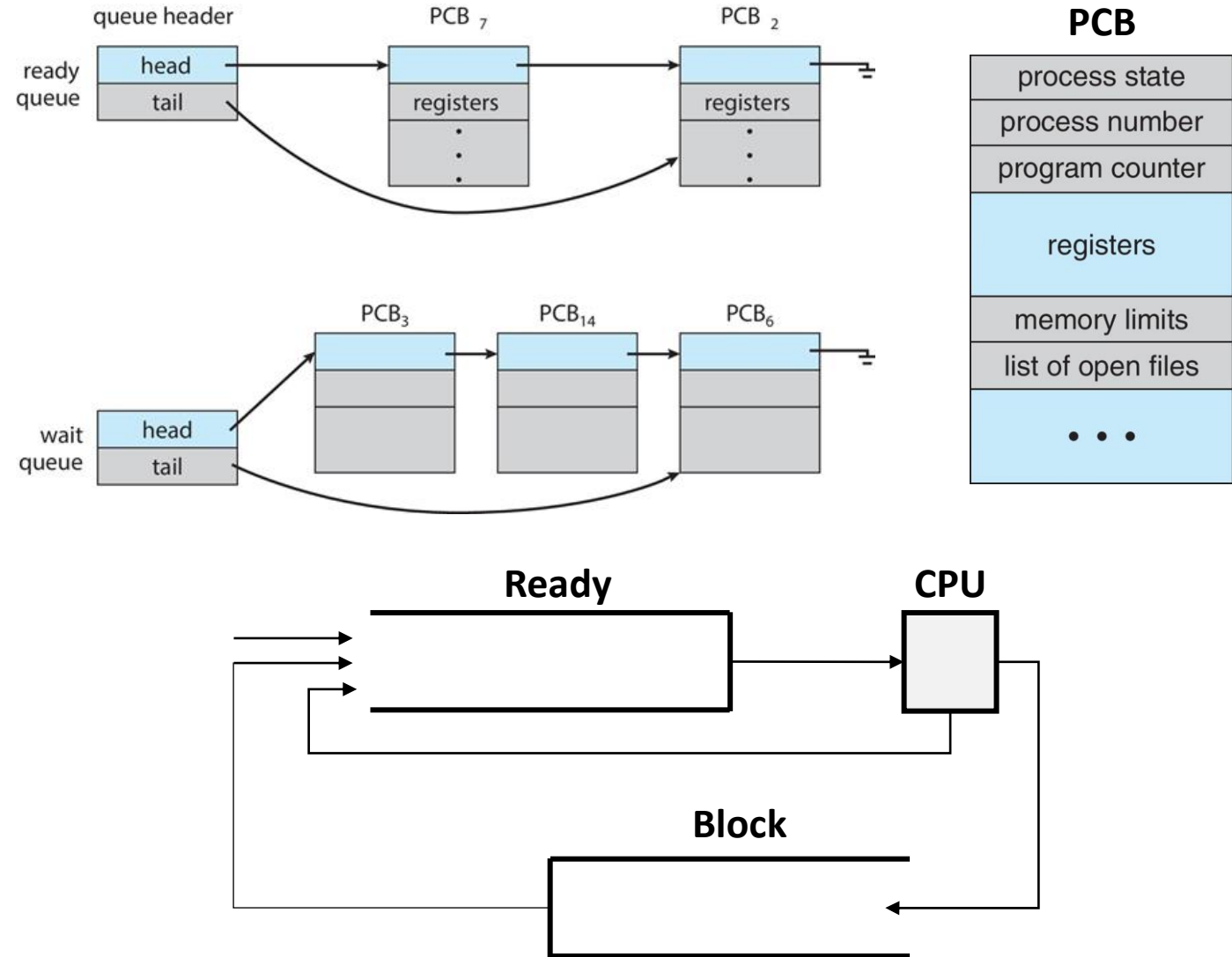
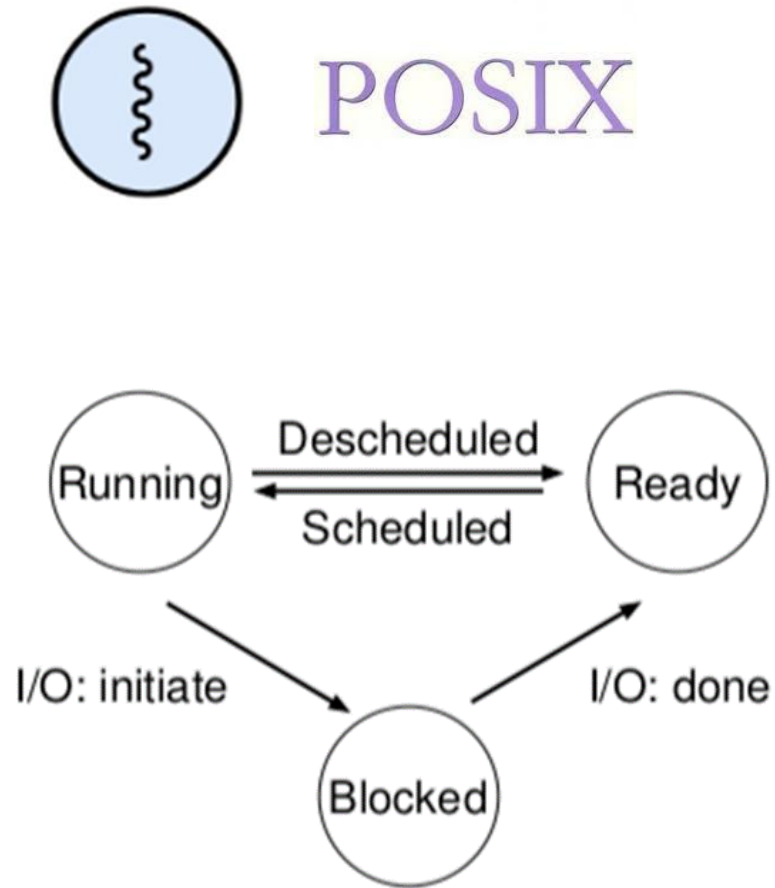
Políticas



Mecanismos



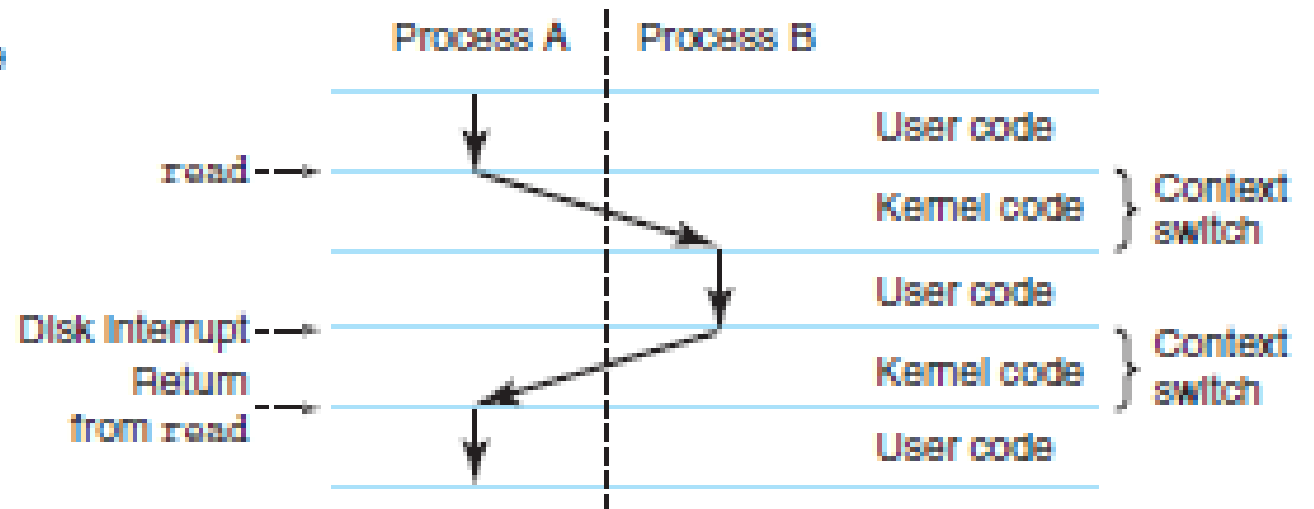
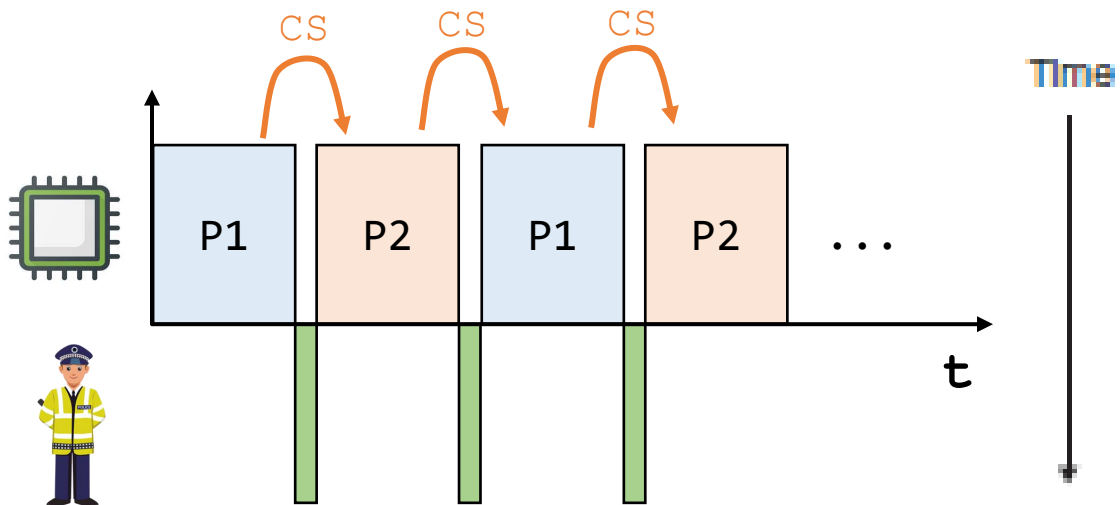
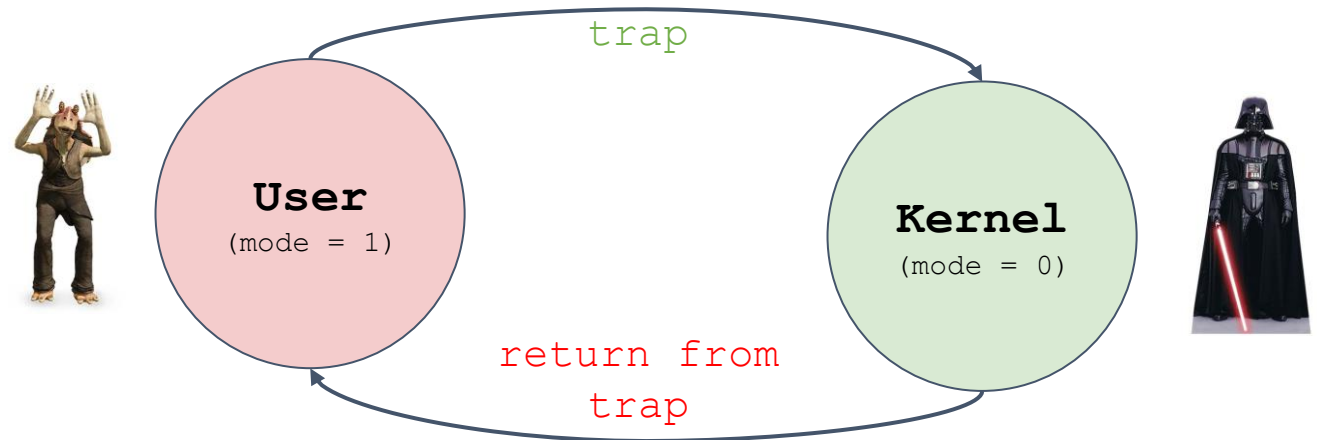
Abstracción CPU - Proceso



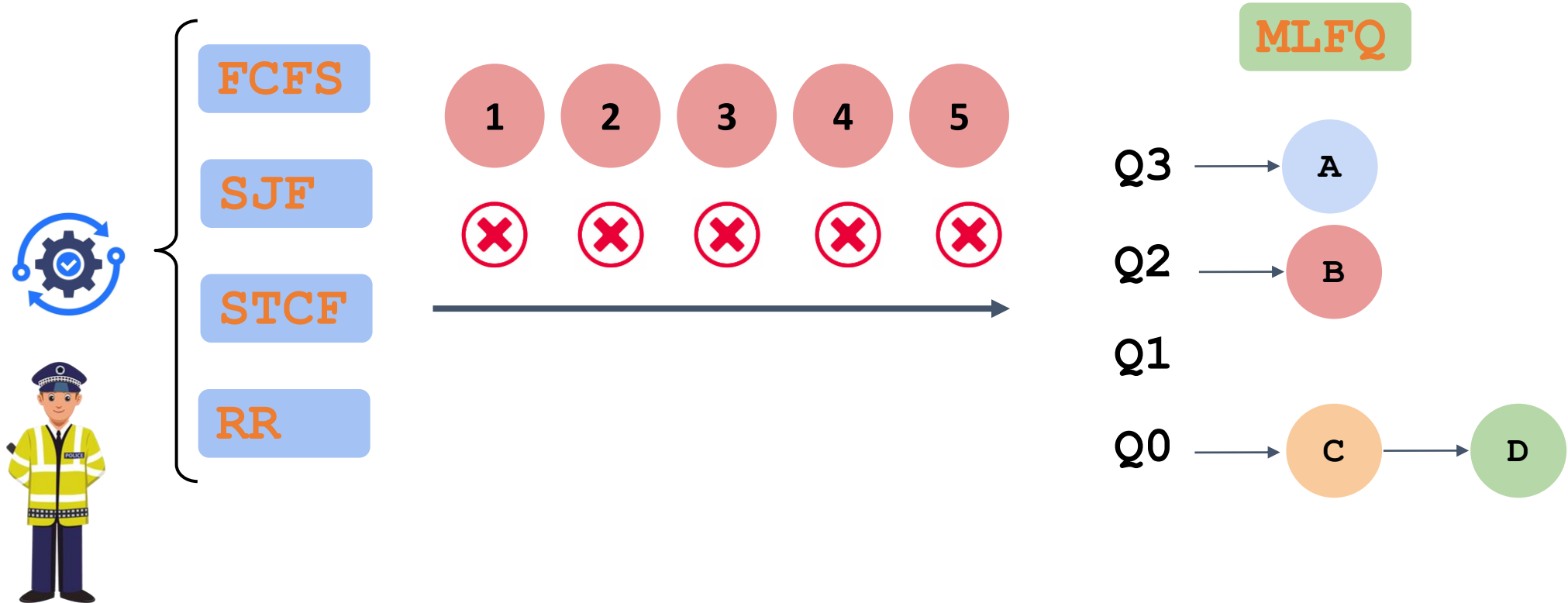
Mecanismos CPU - Proceso

Limited

Direct
Execution

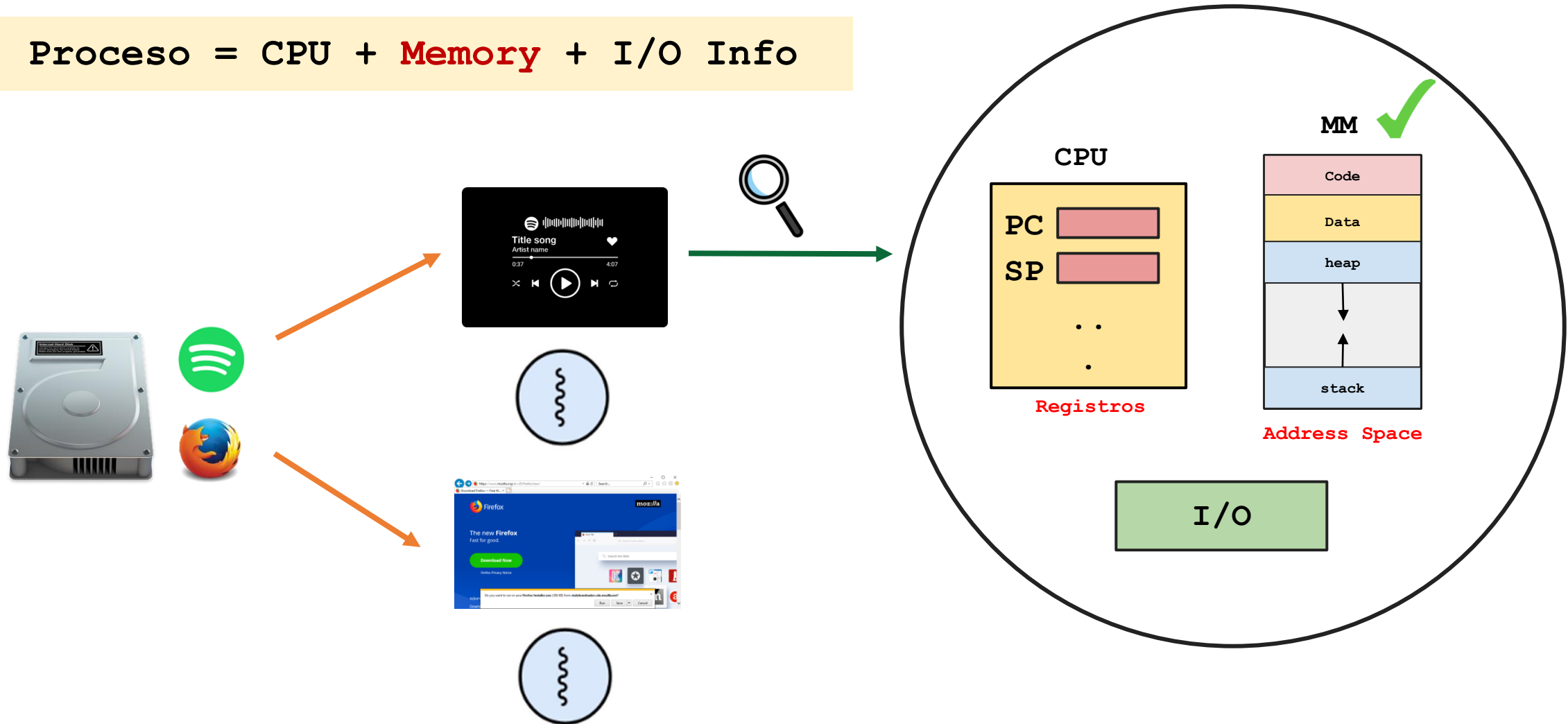


Políticas CPU - Proceso



Virtualización de Memoria - Contextualización

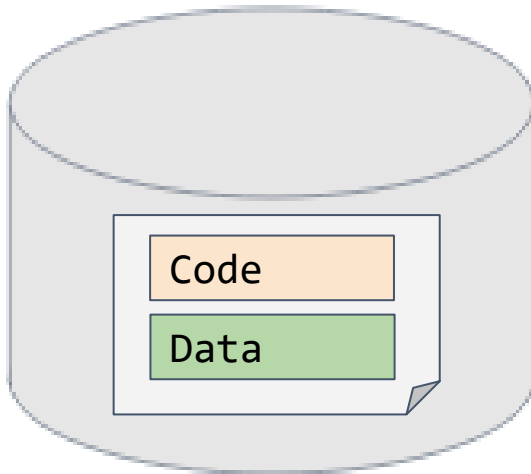
Proceso = CPU + **Memory** + I/O Info



Virtualización de Memoria - Contextualización



mem.c



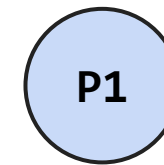
```
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include "common.h"

int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "usage: mem <value>\n");
        exit(1);
    }
    int *p;
    p = malloc(sizeof(int));
    assert(p != NULL);
    printf("(%d) addr pointed to by p: %p\n", (int) getpid(), p);
    *p = atoi(argv[1]); // assign value to addr stored in p
    while (1) {
        Spin(1);
        *p = *p + 1;
        printf("(%d) value of p: %d\n", getpid(), *p);
    }
    return 0;
}
```

Virtualización de Memoria - Contextualización

Pregunta: ¿Cómo funciona el programa?

```
prompt> ./mem 1
(2134) address pointed to by p: 0x200000
(2134) p: 1
(2134) p: 2
(2134) p: 3
(2134) p: 4
(2134) p: 5
^C
```



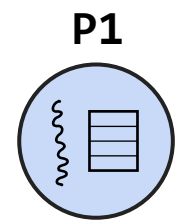
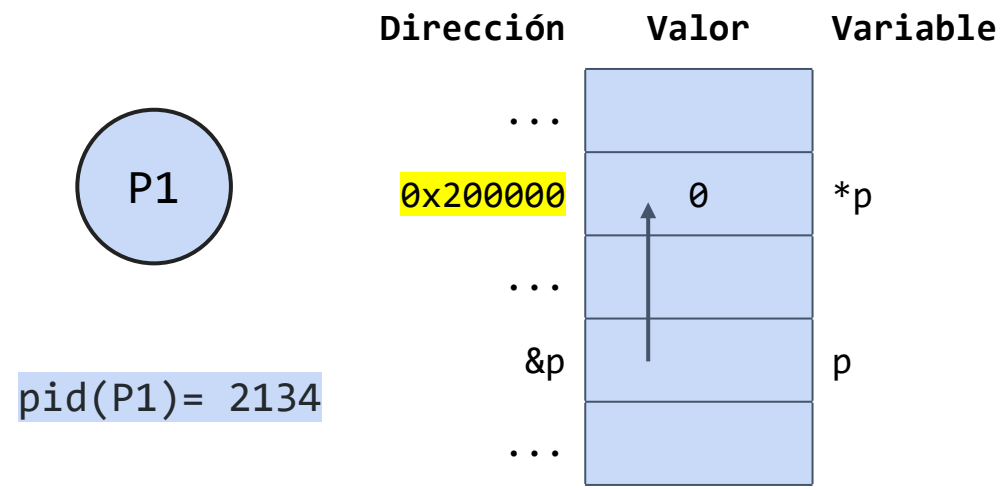
pid(P1)= 2134

Dirección	Valor	Variable
...		
0x200000	0	*p
...		
&p		p
...		

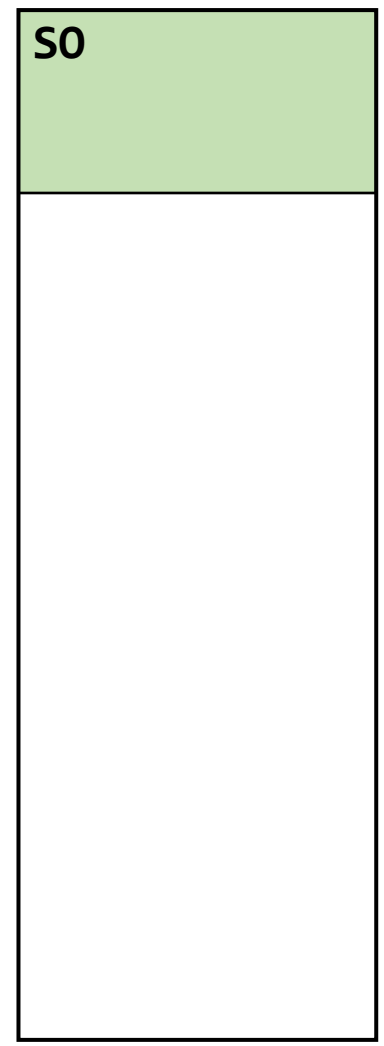
Virtualización de Memoria - Contextualización

Pregunta: ¿Cómo funciona el programa?

```
prompt> ./mem 1
(2134) address pointed to by p: 0x200000
(2134) p: 1
(2134) p: 2
(2134) p: 3
(2134) p: 4
(2134) p: 5
^C
```



Memoria física

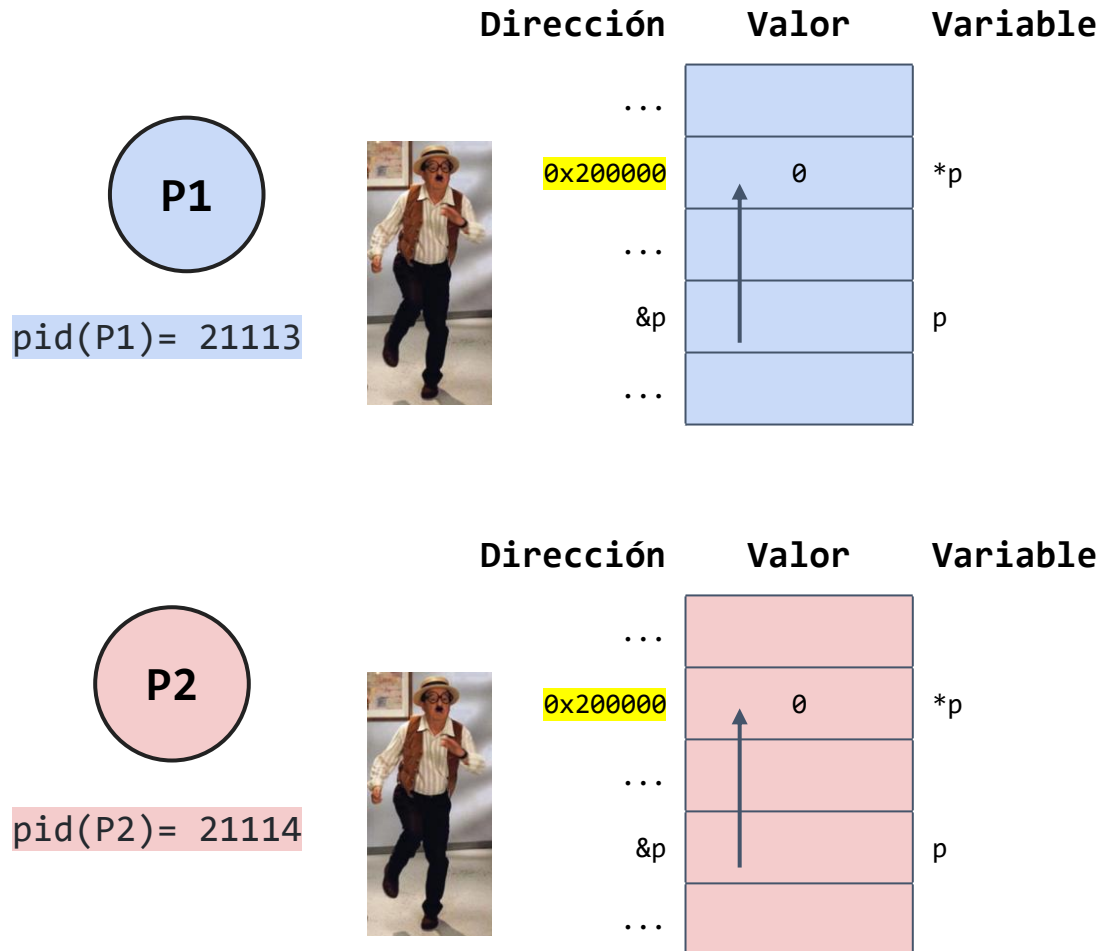


Virtualización de Memoria - Contextualización

Pregunta: ¿Que pasa si dos usuarios corren simultáneamente el programa?

```
prompt> ./mem 1 & ./mem 1 &  
[1] 24113  
[2] 24114  
(24113) address pointed to by p: 0x200000  
(24114) address pointed to by p: 0x200000  
(24113) p: 1  
(24114) p: 1  
(24114) p: 2  
(24113) p: 2  
(24113) p: 3  
(24114) p: 3  
(24113) p: 4  
(24114) p: 4  
...
```

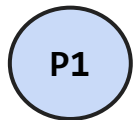
¿Por que son iguales las direcciones si los procesos son diferentes?



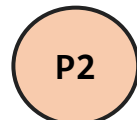
Virtualización de Memoria - Contextualización

Pregunta: ¿Que pasa si dos usuarios corren simultáneamente el programa?

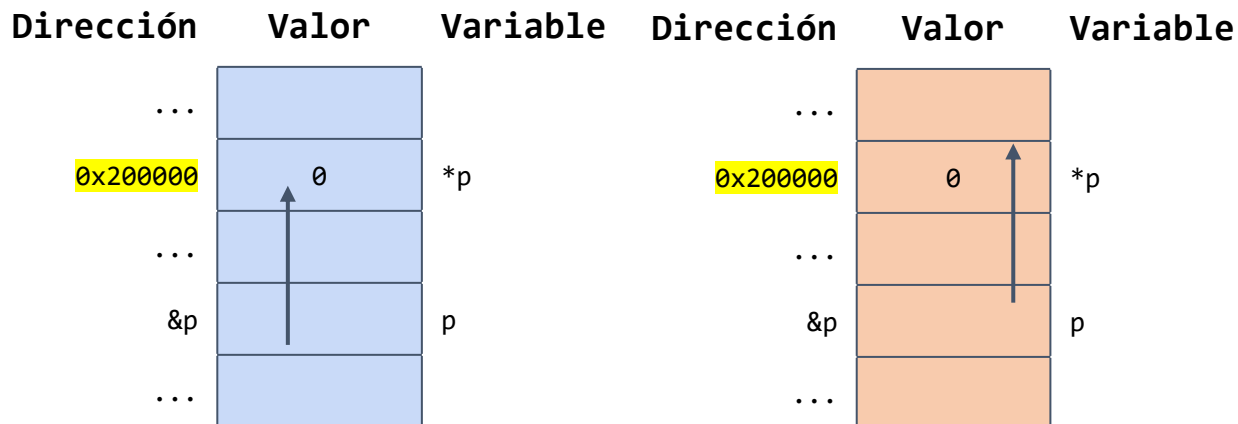
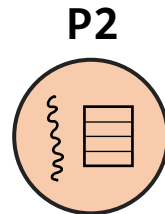
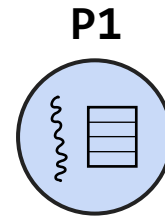
```
prompt> ./mem 1 & ./mem 1 &  
[1] 24113  
[2] 24114  
(24113) address pointed to by p: 0x200000  
(24114) address pointed to by p: 0x200000  
(24113) p: 1  
(24114) p: 1  
(24114) p: 2  
(24113) p: 2  
...
```



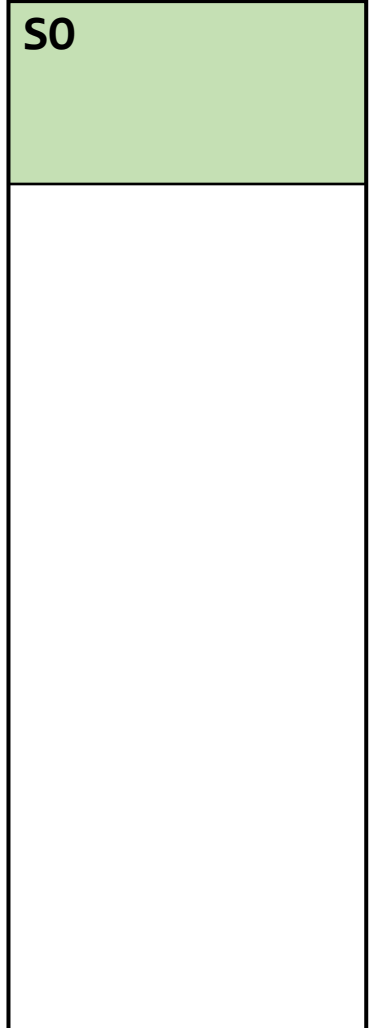
pid(P1)= 21113



pid(P2)= 21114

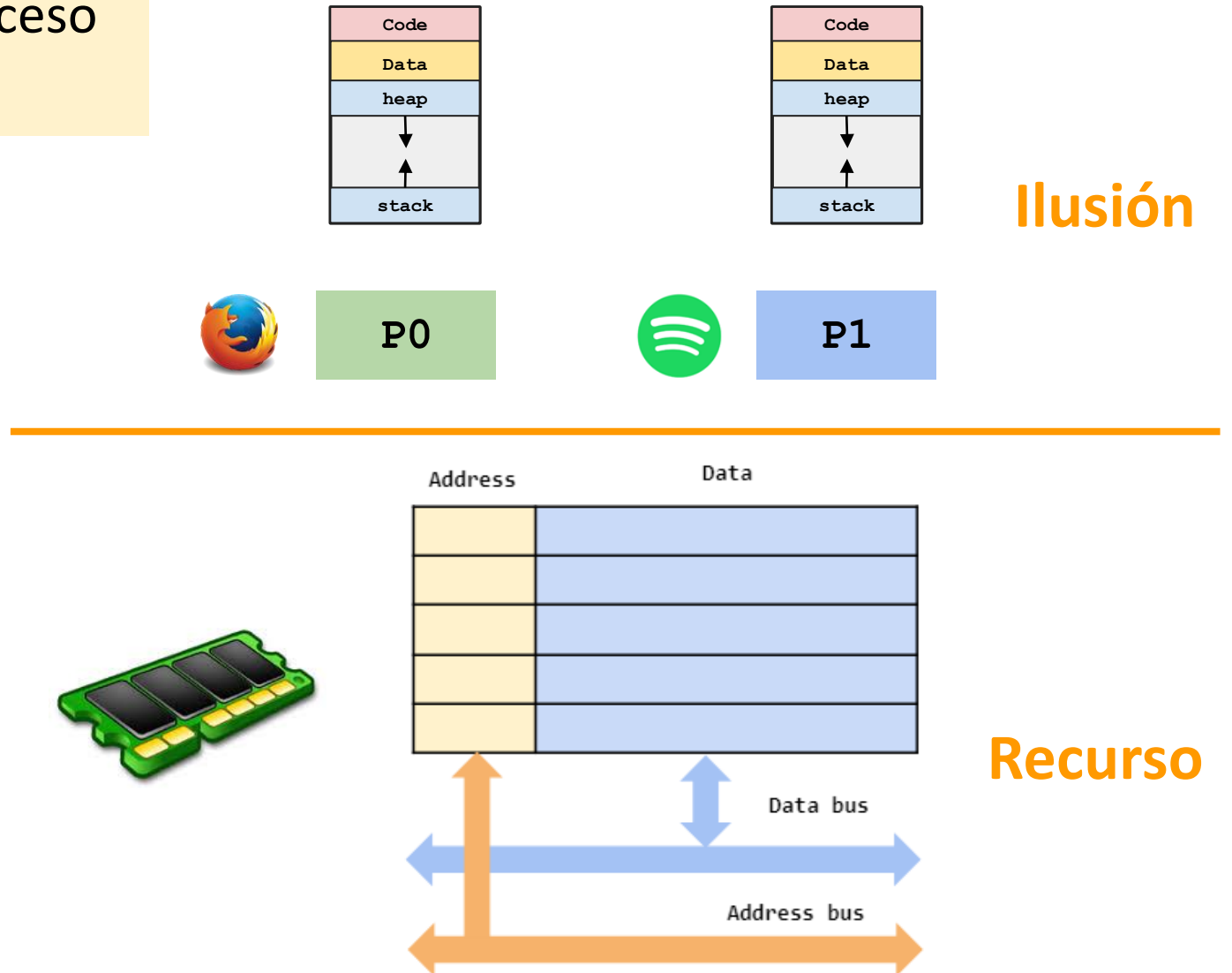
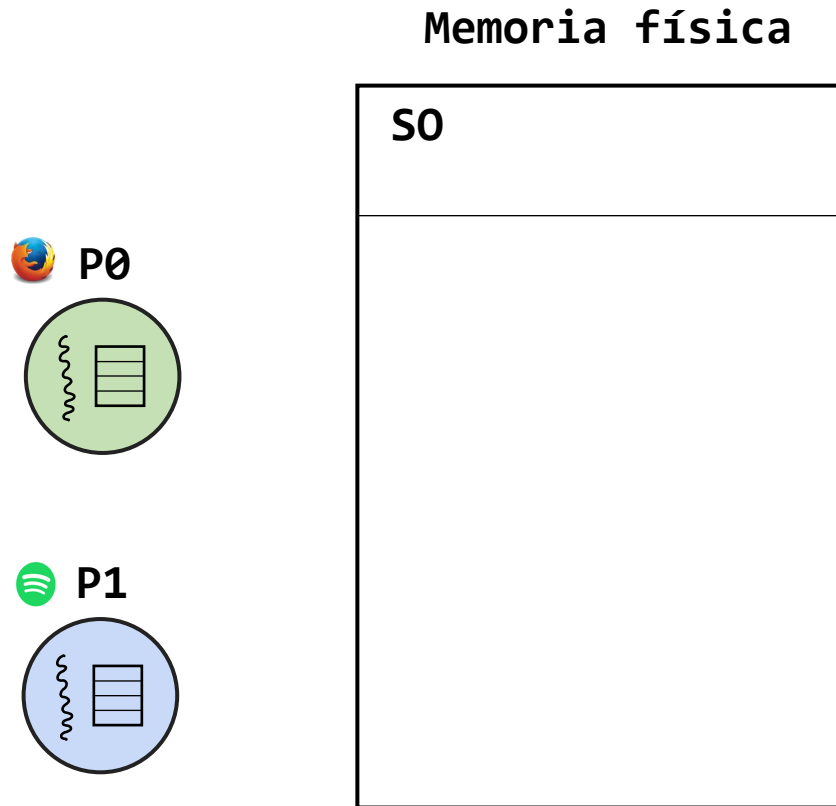


Memoria física



Virtualización de Memoria - Contextualización

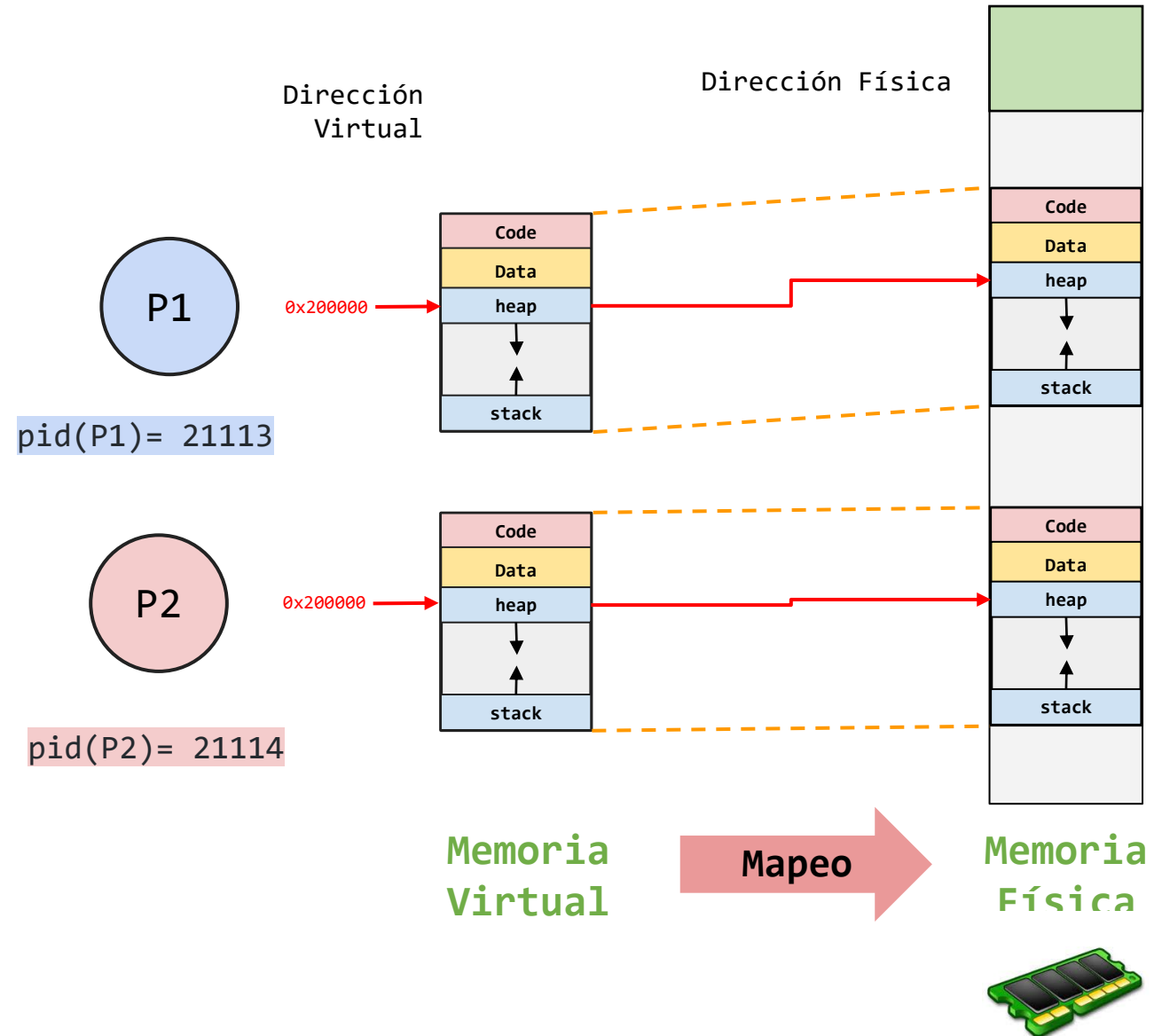
Virtualización de memoria: Cada proceso cree que **tiene su propia memoria**.



Virtualización de Memoria - Contextualización

Virtualización de memoria:
Cada proceso cree que **tiene su propia memoria**.

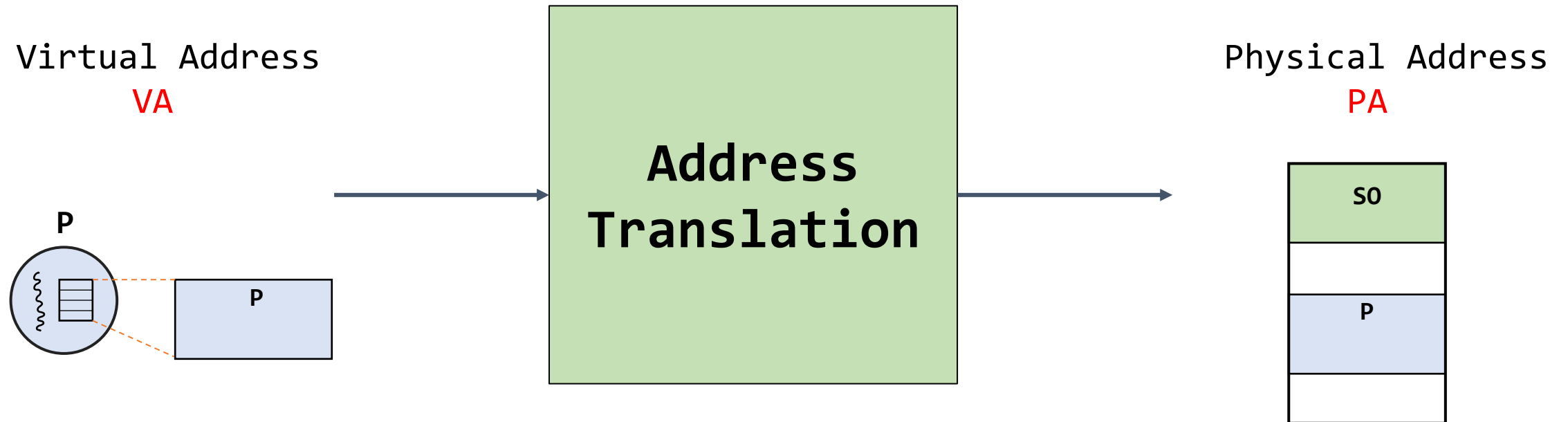
```
prompt> ./mem 1 & ./mem 1 &  
[1] 24113  
[2] 24114  
(24113) address pointed to by p:  
0x200000  
(24114) address pointed to by p:  
0x200000  
(24113) p: 1  
(24114) p: 1  
(24114) p: 2  
(24113) p: 2  
(24113) p: 3  
(24114) p: 3  
(24113) p: 4  
(24114) p: 4  
...
```



Virtualización de Memoria - Contextualización

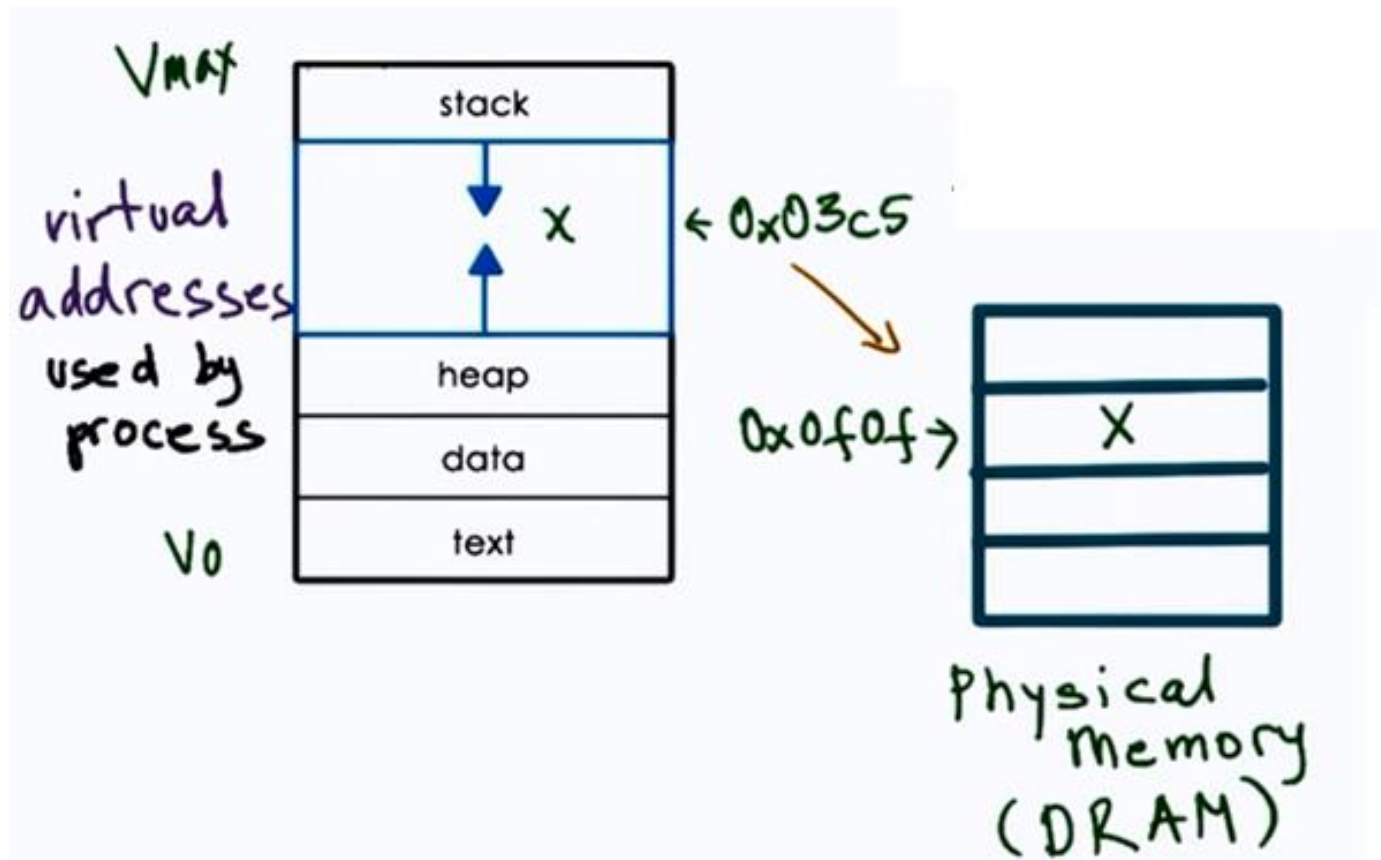
Hardware Address Translation - Address Translation (AT)

- Mecanismo mediante el cual el hardware traduce cada acceso a memoria (Instruction fetch - Load - Store) cambiando la **dirección virtual** proporcionada por la instrucción a una **dirección física** donde la información deseada se encuentra actualmente localizada.
- **Objetivo de la AT:** Dar la ilusión a un programa de que tiene su propio espacio de direcciones privado (propio).



Problema central (crux)

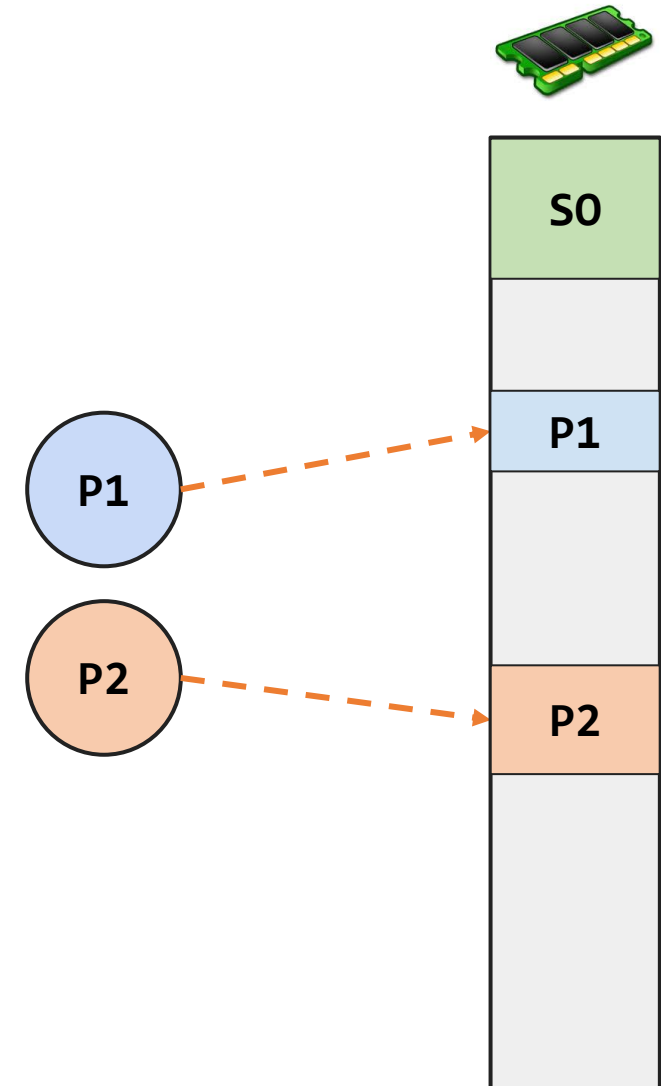
¿Cómo puede el SO construir la ilusión de que cada proceso tiene su propio espacio de memoria privado y grande, si en realidad todos comparten una única memoria física?



Virtualización de Memoria - Contextualización

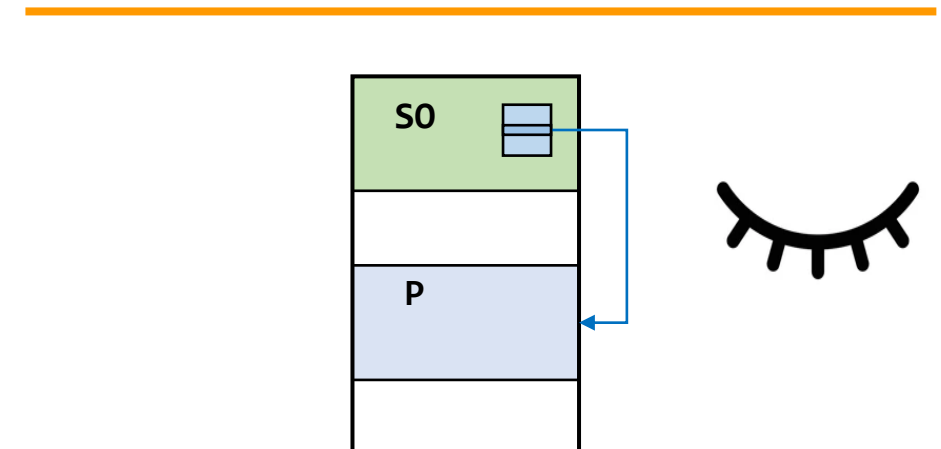
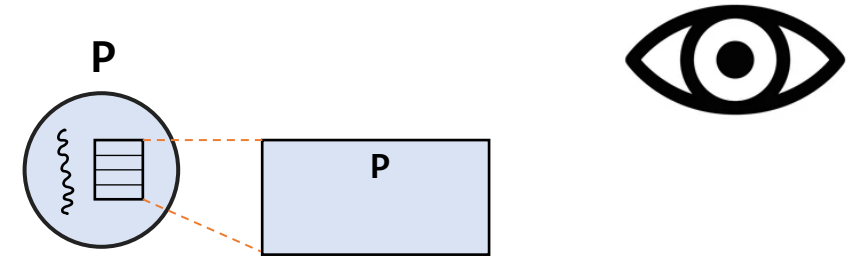
El sistema operativo (SO):

- **Virtualiza** la memoria física.
- **Proporciona una ilusión** para el espacio de memoria de cada proceso.
- Cada proceso **cree** que utiliza todos los recursos de memoria de la máquina.
- El SO **multiplexa** la memoria física **en espacio** (a diferencia de la CPU, que se multiplexa en tiempo): cada proceso recibe simultáneamente una porción distinta de la RAM, no por turnos.



Objetivos de la virtualización de Memoria

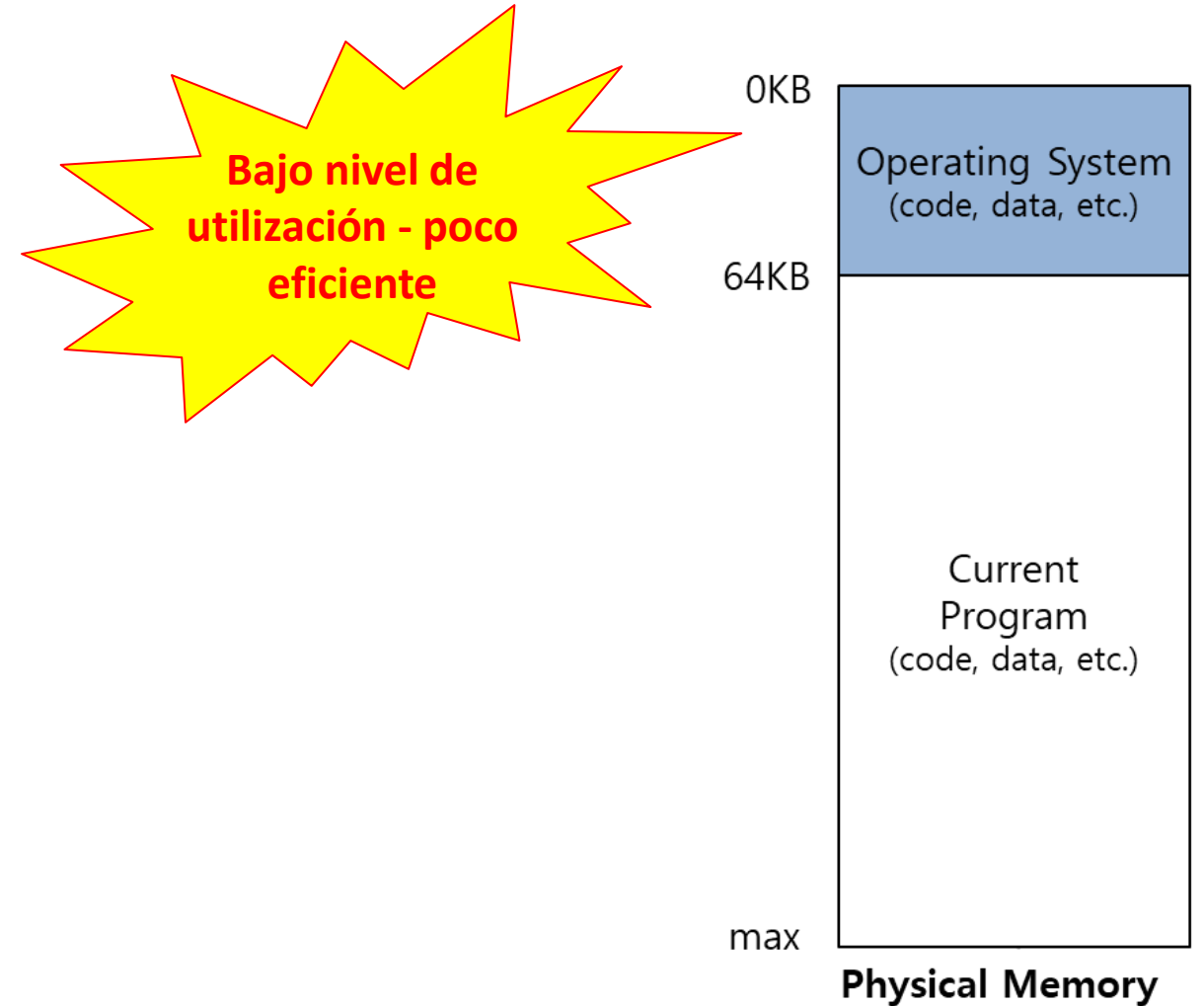
- **Transparencia:** el SO virtualiza la memoria de forma invisible para el programa (el proceso corre como si tuviera toda la memoria física para él solo, sin necesidad de comportarse distinto ni de saber que está siendo virtualizado).
- **Eficiencia** de memoria en términos de **tiempo** y **espacio**.
- **Protección (seguridad)**
 - Garantía de aislamiento entre procesos y SO.
 - Protección ante accesos ilegales de otros procesos.



Sistemas operativos y memoria

Primeros sistemas operativos

- El sistema operativo consistía en un conjunto de rutinas (biblioteca) que se encontraba al inicio de la memoria física.
- Solo se cargaba **un solo proceso** en el resto de la memoria.

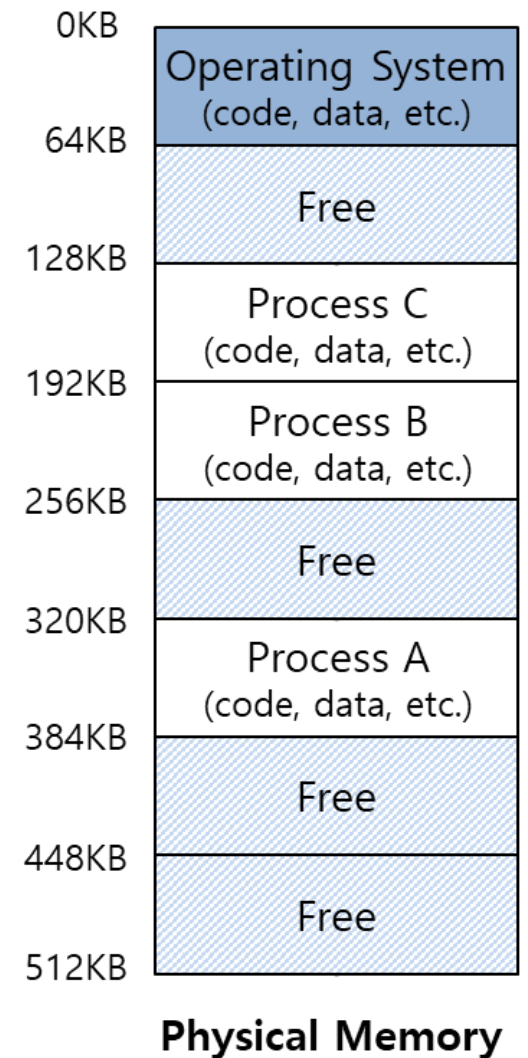


Sistemas operativos y memoria

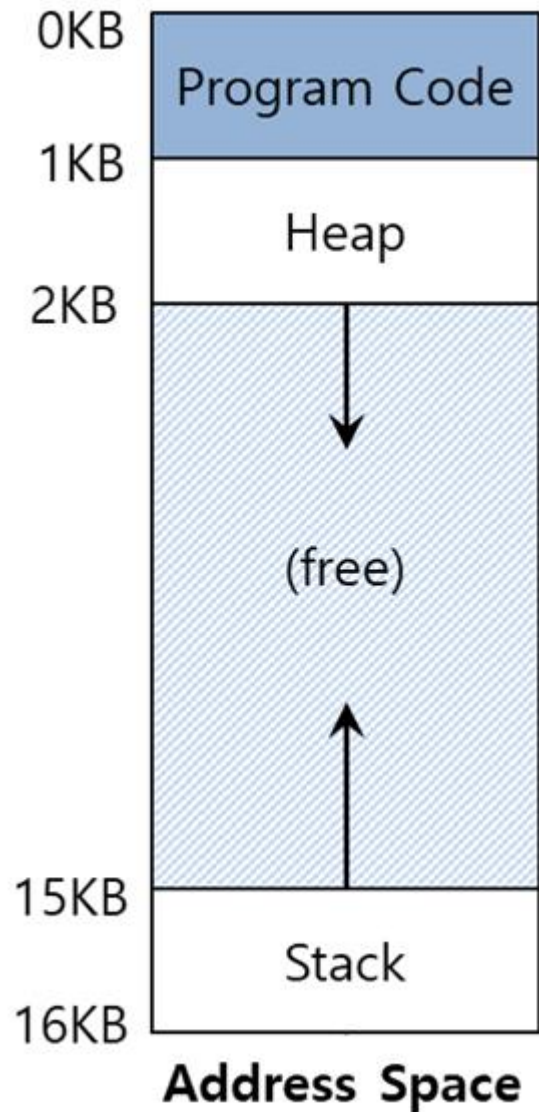
Multiprogramación - tiempo compartido

- Carga múltiples procesos en memoria
 - Ejecuta un proceso por un periodo corto de tiempo.
 - Cambia uso de CPU entre procesos.
 - Incrementa utilización y eficiencia.
- Crea un problema de **protección**.

¡Accesos de
memoria ilegales
de otros
procesos!

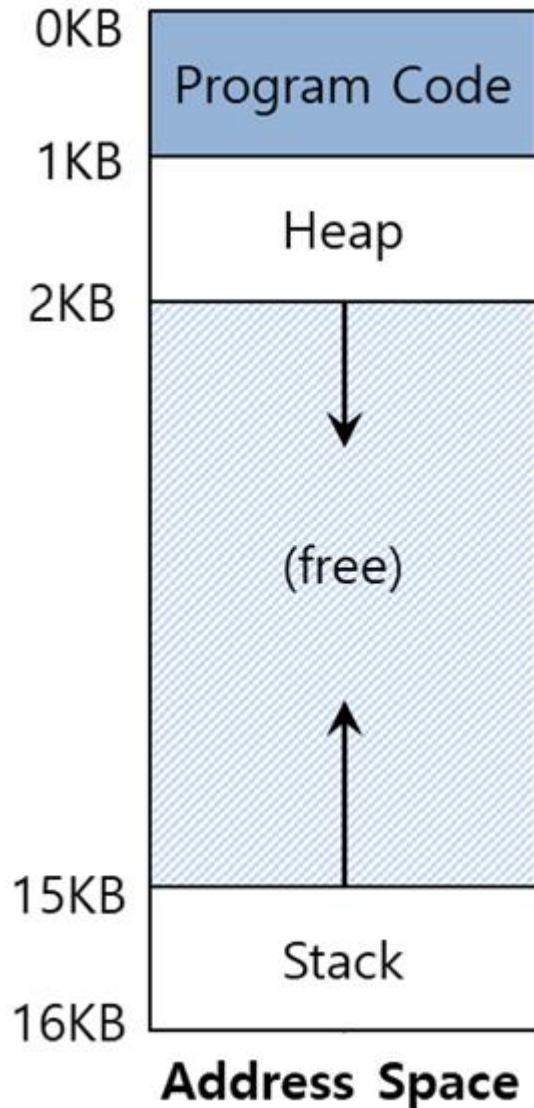


Espacio de direcciones: Address Space



- **Abstracción** de la memoria física creada por el sistema operativo
- Contiene todos los datos e instrucciones de un proceso en ejecución
 - Program code (text)
 - Data
 - Heap
 - Stack

Espacio de direcciones: Address Space



Segmentos del espacio de direcciones

Segmento	Observaciones
Program Code	● Instrucciones ● Variables globales
Heap	● Memoria asignada dinámicamente ○ malloc en C ○ new en Java o C++
Stack	● Memoria automática ● Direcciones y valores de retorno ● Variables locales y argumentos pasados a rutinas

Espacio de direcciones: Address Space

- Todas las direcciones creadas por un programa son virtuales!
- El SO traduce las direcciones **virtuales** a direcciones **físicas**.

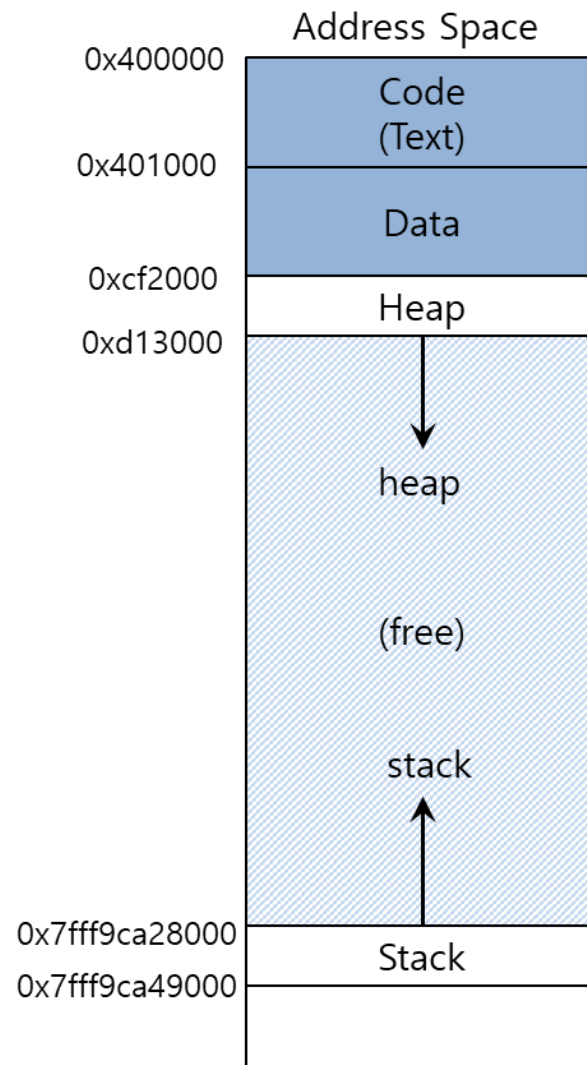
```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    printf("location of code  : %p\n", (void *) main);
    printf("location of heap  : %p\n", (void *) malloc(1));
    int x = 3;
    printf("location of stack : %p\n", (void *) &x);
    return x;
}
```

Ejemplo. [va.c](#) ([simulación](#))

```
location of code  : 0x40057d
location of heap  : 0xcf2010
location of stack : 0x7fff9ca45fcc
```

The output in 64-bit Linux machine



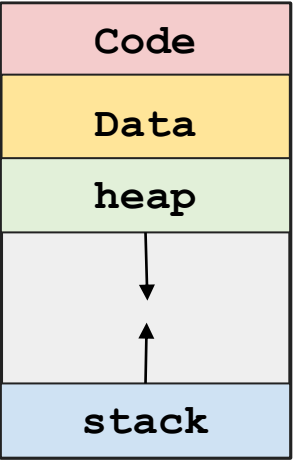
Espacio de direcciones: Address Space

Ejemplo

Dado el siguiente fragmento de código ([link](#)).

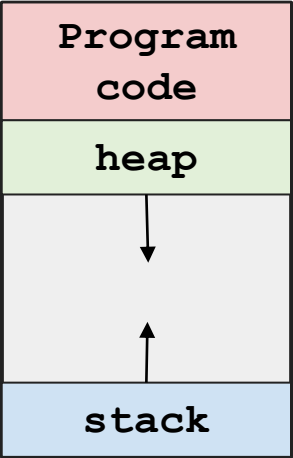
```
int x;  
int main(int argc, char *argv[]) {  
    int y;  
    int *z = malloc(sizeof(int));  
}
```

¿A qué segmento pertenece cada una de las variables mencionadas?



Variable	Location (heap, stack, code)
x	
main	
y	
z	

Ostep



Referencias

- **Operating Systems Three Easy Pieces** ([website](#): Capítulo [13](#))
- Videos de youtube: Clase 1 - Introducción a los sistemas operativos ([link](#)).
- Página de Remzi H. Arpaci-Dusseau ([link](#))
- Operating System Concepts - Tenth Edition ([website](#))
- Operating Systems Notes ([link](#))
- <https://github.com/Aniruddha-Tapas/Operating-Systems-Notes/blob/master/1-Overview.md>
- <https://myaut.github.io/dtrace-stap-book/kernel/proc.html>
- <https://myaut.github.io/dtrace-stap-book/index.html>
- <https://www.technologyuk.net/computing/computer-software/operating-systems/operating-system-utilities.shtml>
- <http://www.it.uu.se/education/course/homepage/os/vt18/module-4/simple-threads/>
- <https://courses.cs.duke.edu/spring19/compsci590.1/slides/>
- <http://csl.snu.ac.kr/courses/4190.307/2020-1/>